

# Welcome to SEAS Online at George Washington University

**Class will begin shortly**

**Audio:** To eliminate background noise, please be sure your audio is muted. To speak, please click the hand icon at the bottom of your screen (**Raise Hand**). When instructor calls on you, click microphone icon to unmute. When you've finished speaking, ***be sure to mute yourself again.***

**Chat:** Please type your questions in Chat.

**Recordings:** As part of the educational support for students, we provide downloadable recordings of each class session to be used exclusively by registered students in that particular class. **Releasing these recordings is strictly prohibited.**

# **SEAS 8515 – Lecture 2**

## **Distributed Computing and Data Storage Systems**

Walid Hassan, M.B.A, D.Eng.

# Traditional Large-Scale Computation

**Early Days:** Processor Bound, and Memory Bound (the CPUs were “slower”...)

**Early Solution:** Bigger Machine... Faster CPUs, More memory

| Aspect           | Mainframes                               | Exadata                                       |
|------------------|------------------------------------------|-----------------------------------------------|
| Era              | Introduced in the 1950s-60s (e.g., IBM). | Introduced in 2008 as a modern solution.      |
| Scalability      | Vertical scaling only.                   | Vertical scaling but integrates modern tech.  |
| Task Diversity   | General-purpose computing.               | Specialized for database workloads.           |
| Technology Stack | Proprietary legacy systems.              | Combines modern hardware and software layers. |

# Distributed Systems

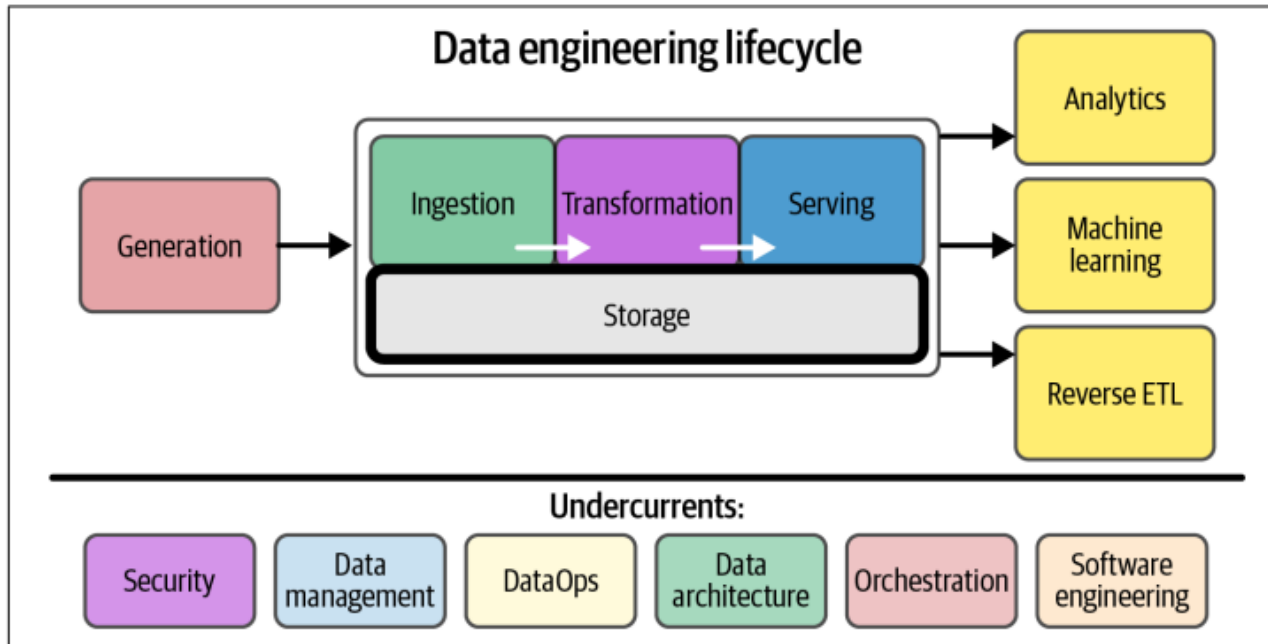
“In pioneer days they used oxen for heavy pulling, and when one ox couldn’t budge a log, we didn’t try to grow a larger ox. We shouldn’t be trying for bigger computers, but for more systems of computers.”

– Grace Hopper



Distributed = many computers tackling a (single) workload.

# Data Engineering Lifecycle



Storage is a key component

# Storage

- You need a place to store data. Choosing a storage solution is key to success in the rest of the data lifecycle, and it's also one of the most complicated stages of the data lifecycle for a variety of reasons.
- First, data architectures in the cloud often leverage several storage solutions.
- Second, few data storage solutions function purely as storage, with many supporting complex transformation queries

# Storage Considerations

## Key engineering considerations

Here are a few key engineering questions to ask when choosing a storage system for a data warehouse, data lakehouse, database, or object storage:

- Is this storage solution compatible with the architecture's required write and read speeds?
- Will storage create a bottleneck for downstream processes?
- Do you understand how this storage technology works? Are you utilizing the storage system optimally or committing unnatural acts
- Will this storage system handle anticipated future scale?

Src: Fundamentals of Data Engineering

© Walid Hassan, M.B.A, D.Eng.

# Storage Considerations

- Will downstream users and processes be able to retrieve data in the required service-level agreement (SLA)?
- Are you capturing metadata about schema evolution, data flows, data lineage, and so forth?
- Is this a pure storage solution (object storage), or does it support complex query patterns (i.e., a cloud data warehouse)?
- Is the storage system schema-agnostic (object storage)? Flexible schema (Cassandra)? Enforced schema (a cloud data warehouse)?
- How are you tracking master data, golden records data quality, and data lineage for data governance?
- How are you handling regulatory compliance and data sovereignty?



# Storage Considerations

## Purpose and use case

- You must first identify the purpose of storing the data. What is it used for? Update patterns.
- Is the abstraction optimized for bulk updates, streaming inserts, or upserts?

## Cost

- What are the direct and indirect financial costs? The time to value? The opportunity costs?

## Separate storage and compute

The trend is toward separating storage and compute, but most systems hybridize separation and colocation.

# Some Hadoop Terminology

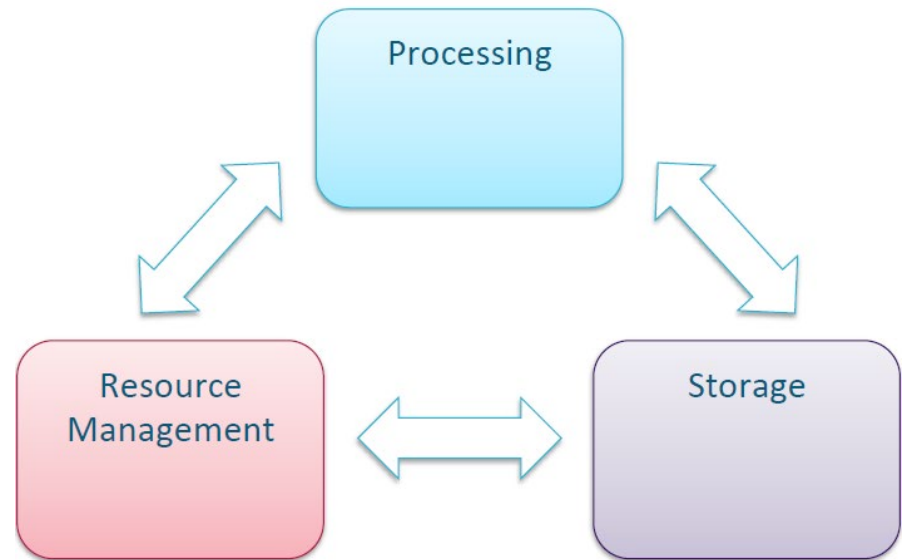
- Cluster: A network of computers collaborating to handle tasks. Functions: Offers data storage, processing power, and resource management.
- Node: A single computer within the cluster.
  - Master Nodes: Coordinate the distribution of tasks and data to worker nodes. The master nodes typically utilize higher quality hardware.
  - The workers consist of virtual machines, running both DataNode and TaskTracker services on commodity hardware, and do the actual work of storing and processing the jobs as directed by the master nodes
  - The final part of the system are the Client Nodes, which are responsible for loading the data and fetching the results.
- Daemon: A background program operating on a node
  - Hadoop Daemons: Each one has a unique role in managing cluster functions

Src:<https://www.databricks.com/glossary/hadoop-cluster>

© Walid Hassan, M.B.A, D.Eng.

# Some Hadoop Terminology

- Hadoop Cluster has 3 main components
  - Storage (HDFS)
  - Resource Manager (YARN)
  - Processing (Map Reduce)



# Key Hadoop Assumptions

- Large Data Sets:
  - A typical file in HDFS is gigabytes to terabytes in size.
  - It should provide high aggregate data bandwidth and scale to hundreds of nodes in a single cluster.
  - It should support tens of millions of files in a single instance.
- Simple Coherency Model:
  - HDFS applications need a write-once-read-many access model for files
  - No Random write are allowed.
  - A MapReduce application or a web crawler application fits perfectly with this model. ~~There is a plan to support appending writes to files in the future.~~

Src:[https://hadoop.apache.org/docs/r1.2.1/hdfs\\_design.html](https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html)

# Key Hadoop Assumptions

“Moving Computation is Cheaper than Moving Data”

- A computation requested by an application is much more efficient if it is executed near the data it operates on (especially when data set is huge).
- The assumption is that it is often better to migrate the computation closer to where the data is located rather than moving the data to where the application is running. HDFS provides interfaces for applications to move themselves closer to where the data is located.

Src:[https://hadoop.apache.org/docs/r1.2.1/hdfs\\_design.html](https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html)

© Walid Hassan, M.B.A, D.Eng.

# What is Hadoop?

- The Hadoop Distributed File System (HDFS) is a distributed file system designed to run on commodity hardware.
- The key differences from other distributed file systems are significant.
  - HDFS is highly fault-tolerant and is designed to be deployed on low-cost hardware.
  - HDFS provides high throughput access to application data and is suitable for applications that have large data sets.
- Hadoop is written in Java and operates on top of the underlying file system.

Src:[https://hadoop.apache.org/docs/r1.2.1/hdfs\\_design.html](https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html)

© Walid Hassan, M.B.A, D.Eng.

# What is Hadoop?

Hadoop

Operating System

Storage



Src:[https://hadoop.apache.org/docs/r1.2.1/hdfs\\_design.html](https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html)

© Walid Hassan, M.B.A, D.Eng.

# NameSpace/Data Node

- HDFS has a master/slave architecture.
- An HDFS cluster consists of
  - a single NameNode, a master server that manages the file system namespace and regulates access to files by clients.
  - A number of DataNodes, usually one per node in the cluster, which manage storage attached to the nodes that they run on.
- HDFS exposes a file system namespace and allows user data to be stored in files.
- Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.

Src:[https://hadoop.apache.org/docs/r1.2.1/hdfs\\_design.html](https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html)



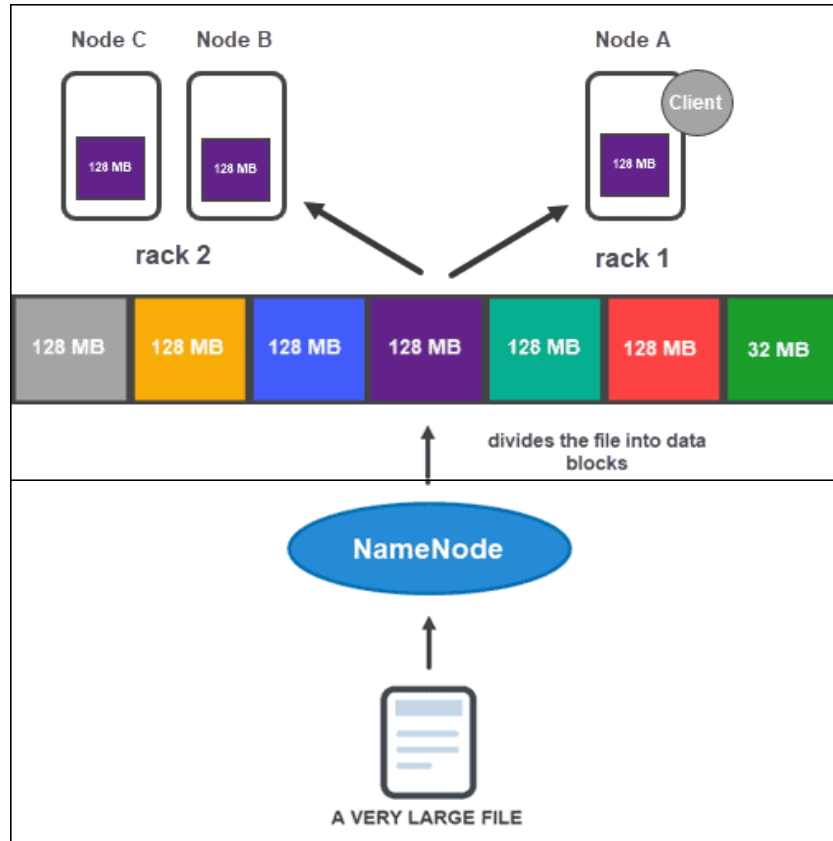
# Namespace/Data Node

- **NameNode**: executes file system namespace operations like opening, closing, and renaming files and directories. It also determines the mapping of blocks to DataNodes.
- **DataNodes**: are responsible for serving read and write requests from the file system's clients. The DataNodes also perform block creation, deletion, and replication upon instruction from the NameNode

Src:[https://hadoop.apache.org/docs/r1.2.1/hdfs\\_design.html](https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html)

© Walid Hassan, M.B.A, D.Eng.

# NameSpace/Data Node

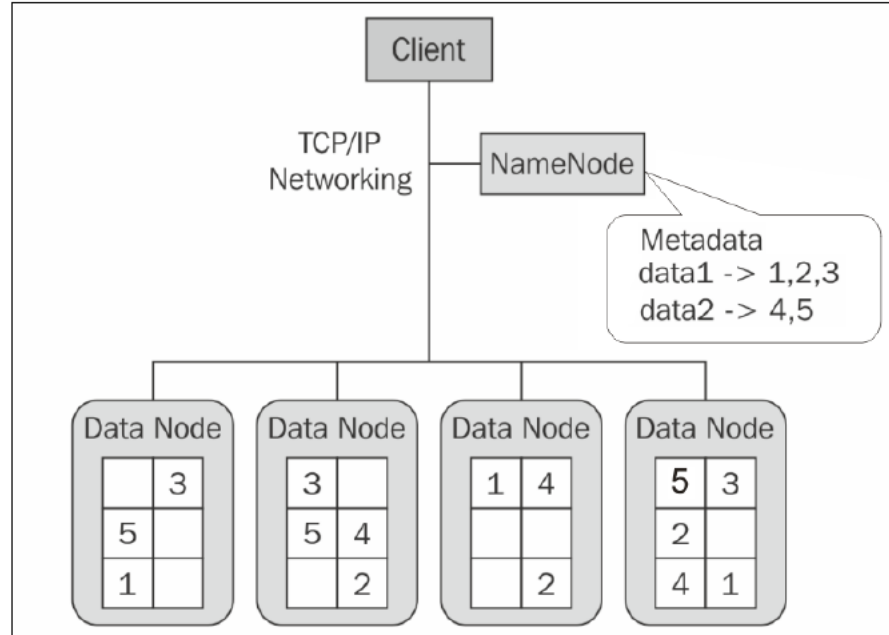


- In HDFS, data is split into blocks (128MB each)
- The NameNode contains metadata about files and blocks, as well as the location of each data block. Once the client knows the location, it interacts directly with the DataNode.

Src: <https://phoenixnap.com/kb/what-is-hdfs>

© Walid Hassan, M.B.A, D.Eng.

# Rack Awareness & Replication

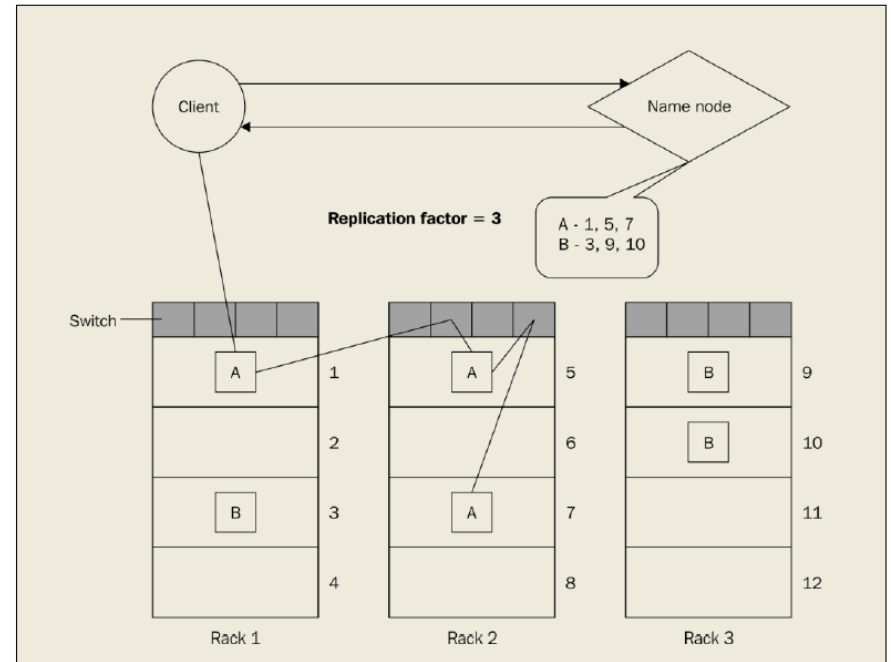


Src: Hadoop Explained, Aravind Shenoy

© Walid Hassan, M.B.A, D.Eng.

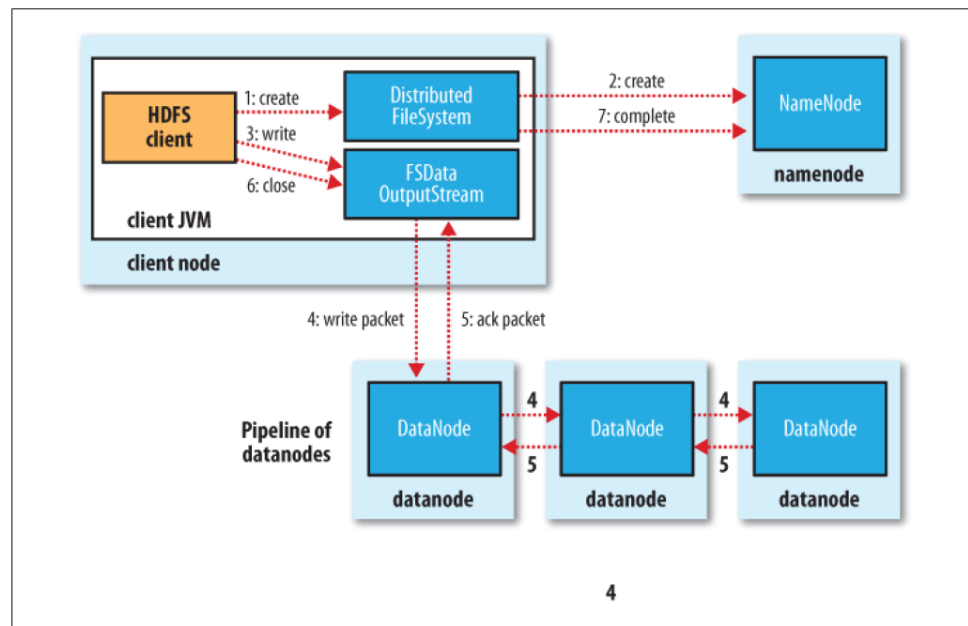
# Rack Awareness & Replication

- Rack 1, Rack 2, and Rack 3. Rack 1 has DataNodes 1, 2, 3, and 4. Rack 2 has DataNodes 5, 6, 7, and 8. Rack 3 has DataNodes 9, 10, 11, and 12.
- Rack awareness is an imperative feature in HDFS. The replication factor used is 3. Hence, the data will be stored on three DataNodes. Data block A is stored on DataNodes 1, 5, and 7. The client will contact the NameNode to get the location of the data. From the NameNode, the client gets the information that the data is stored on DataNode 1 on Rack 1 and DataNodes 5 and 7 on Rack 2.



# Anatomy of an HDFS write

- Step1: Clients Calls Create
- 2: Namenode is contacted (RPC) to create file (many checks are done to ensure does not exists...etc). If All good, client is given an ouptutstream.
- 3: OutputStream splits the data into packets (coordinates the communications with data nodes and Namenode which picks up the datanodes to use for writing packets).
- 4. Datanodes write and forward the packets to the replica nodes.
- 5, acknowledgement of packets being written, 6, file is closed, and namenode is notified

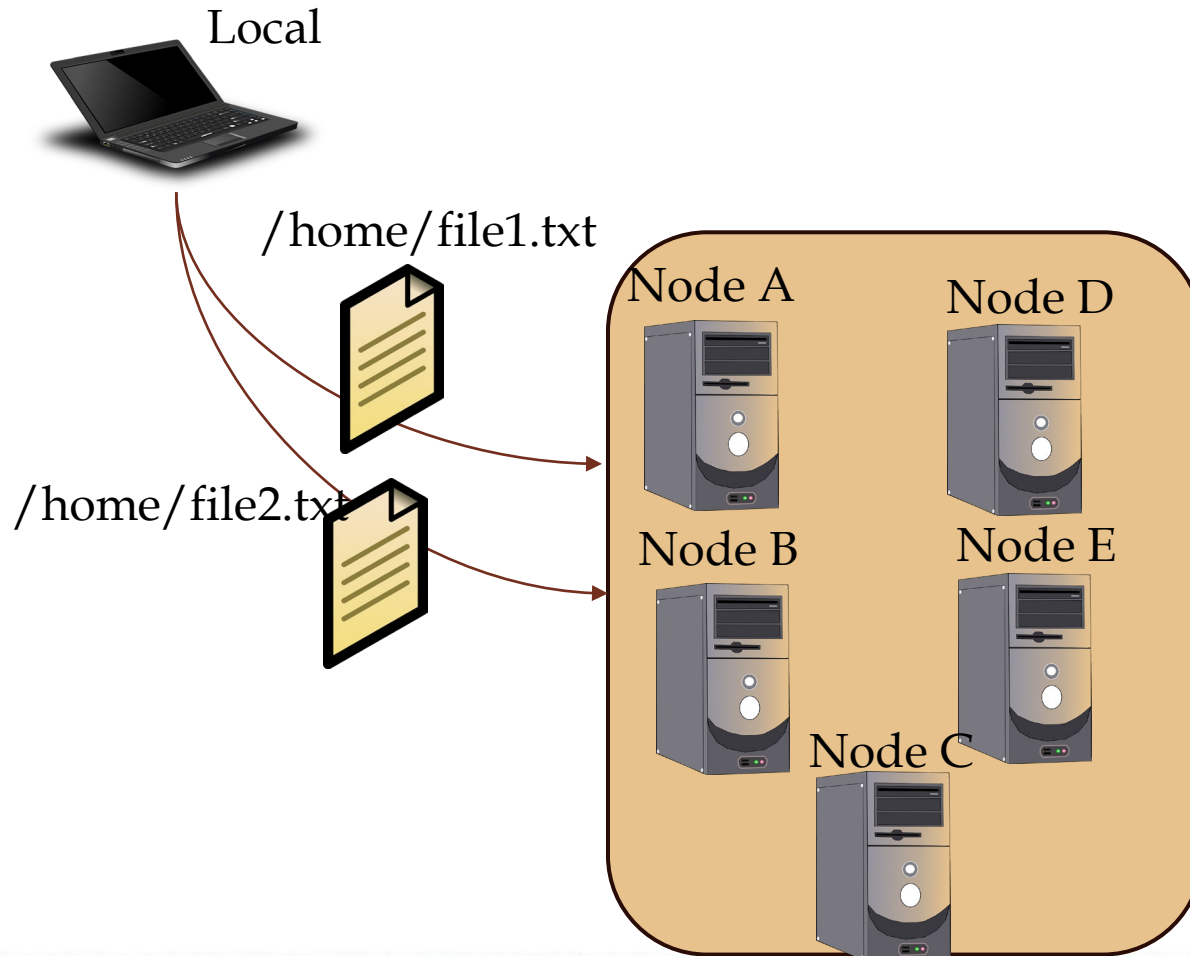


4

Src:: Hadoop The definitive Guide, Tim White, O'Reilly

© Walid Hassan, M.B.A, D.Eng.

# Anatomy of an HDFS write



Src.: Hadoop The definitive Guide, Tim White, O'Reilly

© Walid Hassan, M.B.A, D.Eng.

# Anatomy of an HDFS write

Local



Metadata:

/home/file1.txt: B1,B2,B3  
/home/file2.txt: B4,B5

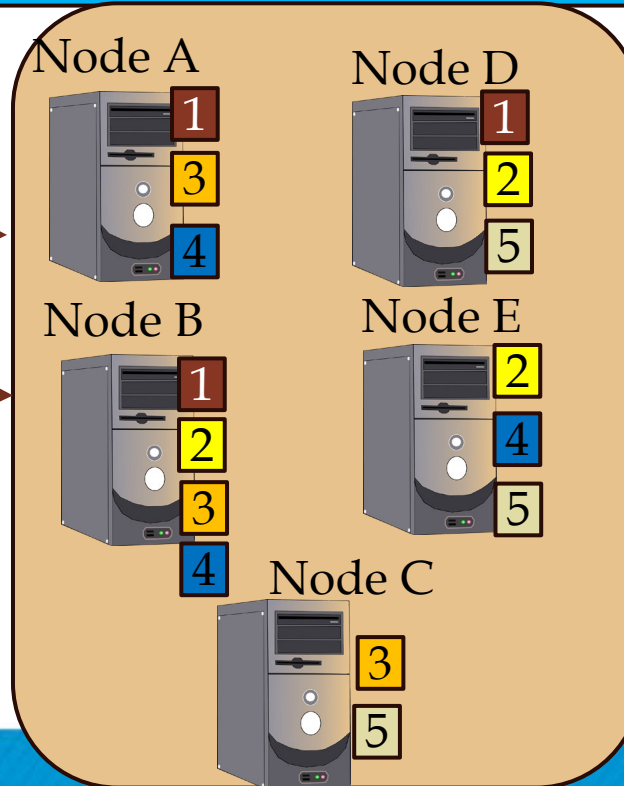
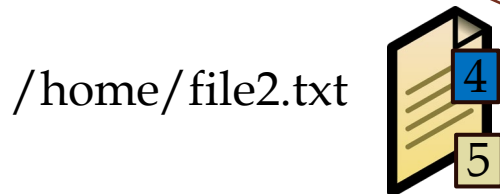
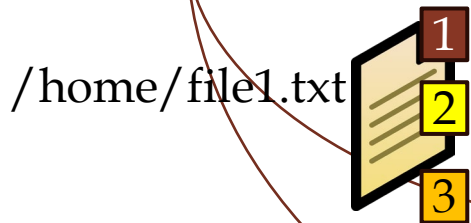
B1: A, B, D

B2: B,D,E

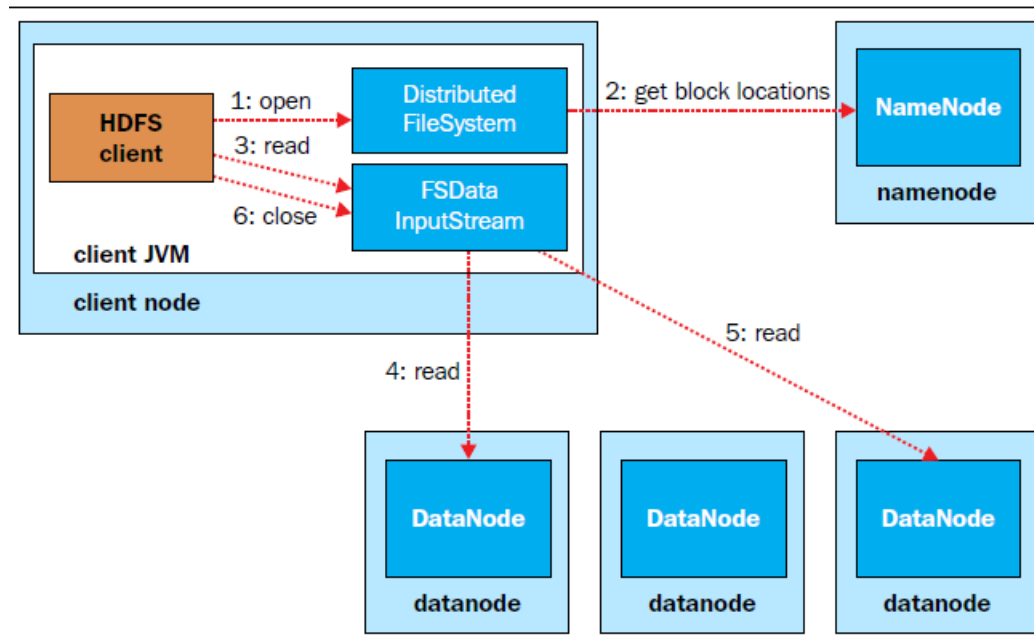
B3: A,B,C

B4: A,B,E

B5: C,E,D



# Read Operation





# Anatomy of an HDFS Read

Local

Metadata:

/home/file1.txt: B1,B2,B3  
/home/file2.txt: B4,B5

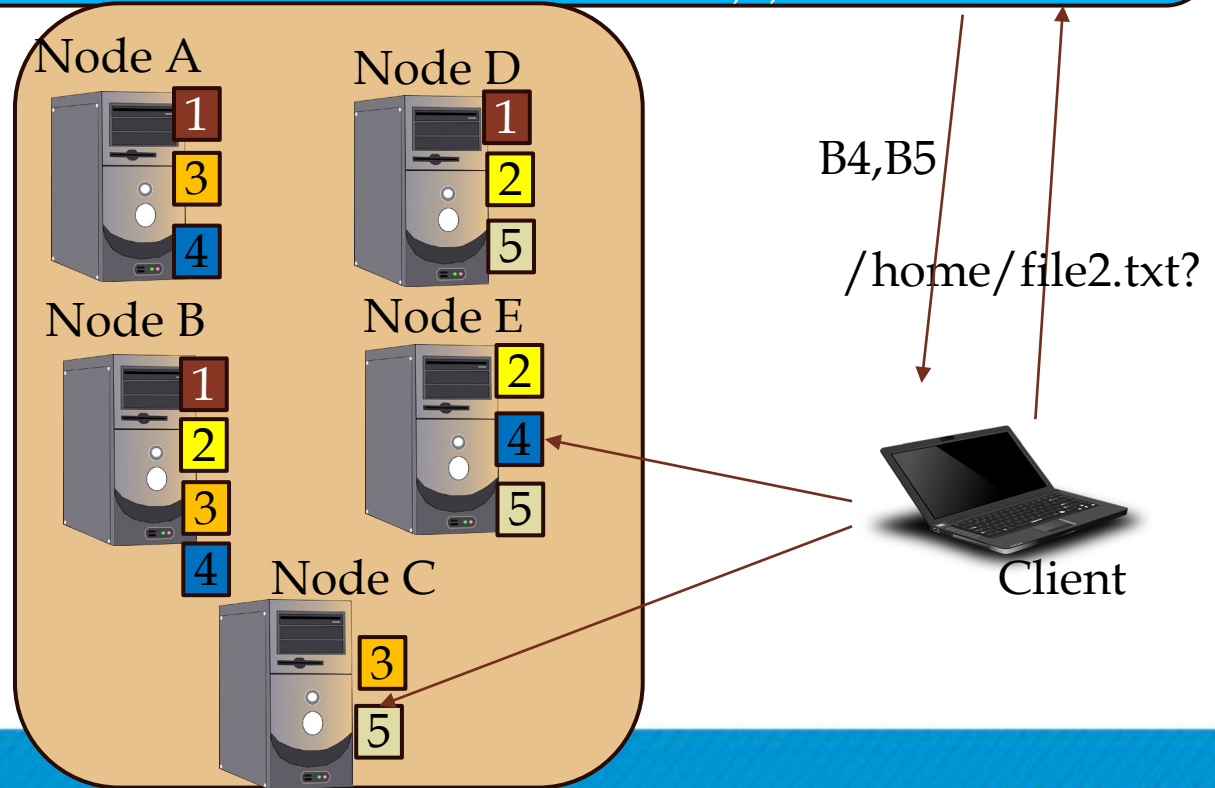
B1: A, B, D

B2: B,D,E

B3: A,B,C

B4: A,B,E

B5: C,E,D



# Racks

Racks are groups of servers (nodes) organized physically and connected through a switch. Rack awareness enables Hadoop to:

- **Optimize Data Placement:** By storing copies of data on different racks, Hadoop minimizes data loss risk in case of rack failure, enhancing data reliability.
- **Improve Network Efficiency:** Hadoop tries to place data copies on nodes within the same rack to reduce cross-rack data transfer, improving data access speed and reducing bandwidth usage.
- **Increase Fault Tolerance:** If a rack goes offline, data is still accessible from replicas on other racks.

# Replication

The replication factor specifies the number of copies (or replicas) of each data block stored in the cluster. When placing these replicas, Hadoop typically follows a strategy like this:

- First Replica: Placed on the same node where the data is written (local node, if possible).
- Second Replica: Placed on a different node in a different rack, ensuring fault tolerance across racks.
- Third Replica: Often placed on a different node within the same rack as the second replica.

# Interaction with HDFS

- HDFS allows user data to be organized in the form of files and directories.
- It provides a command line interface called FS shell that lets a user interact with the data in HDFS (similar to other shells e.g. bash, csh)

| Action                                               | Command                                |
|------------------------------------------------------|----------------------------------------|
| Create a directory named /foodir                     | bin/hadoop dfs -mkdir /foodir          |
| Remove a directory named /foodir                     | bin/hadoop dfs -rmr /foodir            |
| View the contents of a file named /foodir/myfile.txt | bin/hadoop dfs -cat /foodir/myfile.txt |

Src:[https://hadoop.apache.org/docs/r1.2.1/hdfs\\_design.html](https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html)

# Guidelines for HDFS

## Use a logical directory structure:

Create separate directories for raw data, processed data, and output data1.

Use a structure like /raw/businessarea/sourcesystem/data/ and /raw/businessarea/sourcesystem/meta/ for raw data1.

For processed data, use /standard/businessarea/sourcesystem/data/ and /standard/businessarea/sourcesystem/meta/:

/user for user local activities...etc

## Implement data lifecycle management:

Archive or delete old data that is no longer needed.

Use time-based retention policies.

Src:[https://hadoop.apache.org/docs/r1.2.1/hdfs\\_design.html](https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html)

© Walid Hassan, M.B.A, D.Eng.

# Guidelines for HDFS

## **Use meaningful naming conventions:**

Adopt clear, consistent naming for directories and files.

Include metadata like dates in filenames where appropriate.

## **Implement proper access controls:**

Use HDFS permissions and ACLs to control data access.

Organize data to facilitate setting appropriate permissions.

## **Plan for scalability:**

Design your directory structure to accommodate future growth.

Avoid designs that might lead to hotspots as data volumes increase.

Src:[https://hadoop.apache.org/docs/r1.2.1/hdfs\\_design.html](https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html)

© Walid Hassan, M.B.A, D.Eng.



Hands On.....

Databricks file commands

DBUtils: <https://docs.databricks.com/en/dev-tools/databricks-utils.html>

# HDFS Key Features

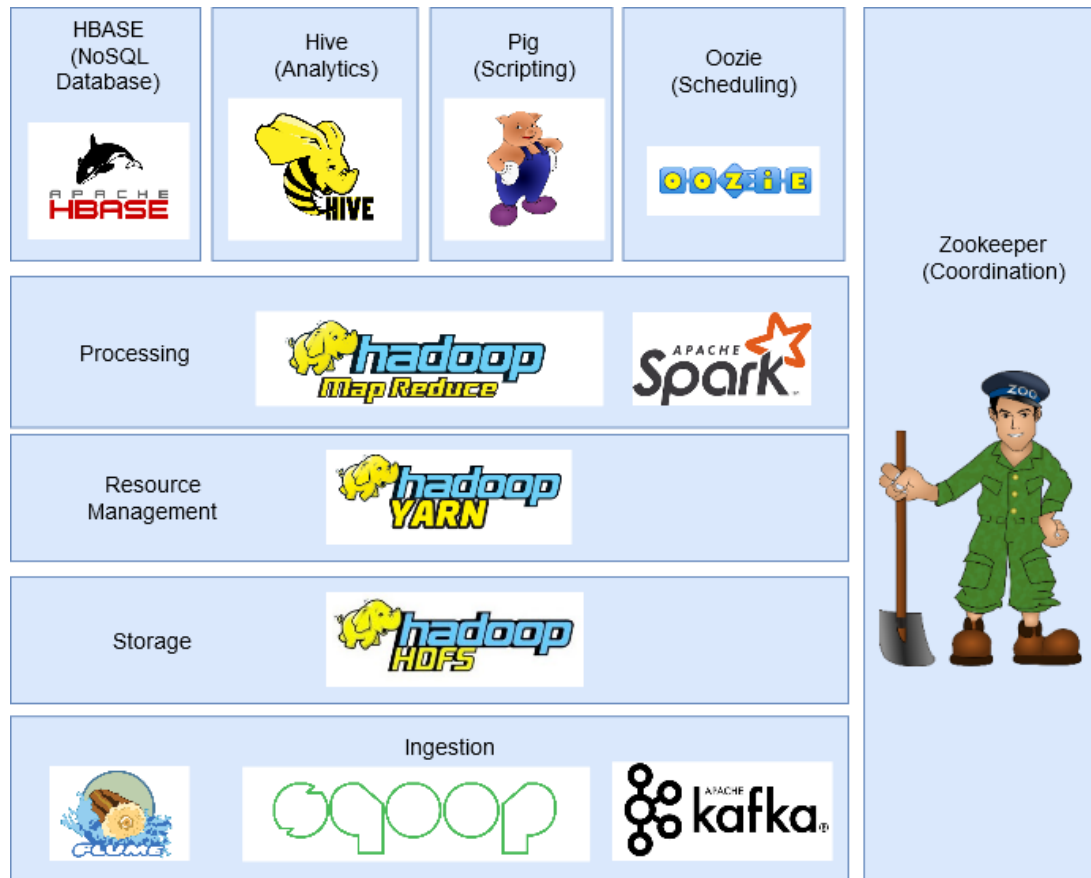
- **Scalability:** HDFS can store and manage data sets that are in the petabyte range, allowing organizations to efficiently store and manage large amounts of data.
- **Fault tolerance:** HDFS provides fault tolerance by replicating data across multiple nodes in a Hadoop cluster. In case of node failures, the system can still access the data by reading from the replicated copies.
- **High availability:** HDFS provides high availability by automatically detecting node failures and transferring the data to other nodes to ensure uninterrupted access to the data.

Src: <https://www.codewithfaraz.com/article/78/top-15-hadoop-ecosystem-components-in-2024-a-comprehensive-guide>

© Walid Hassan, M.B.A, D.Eng.



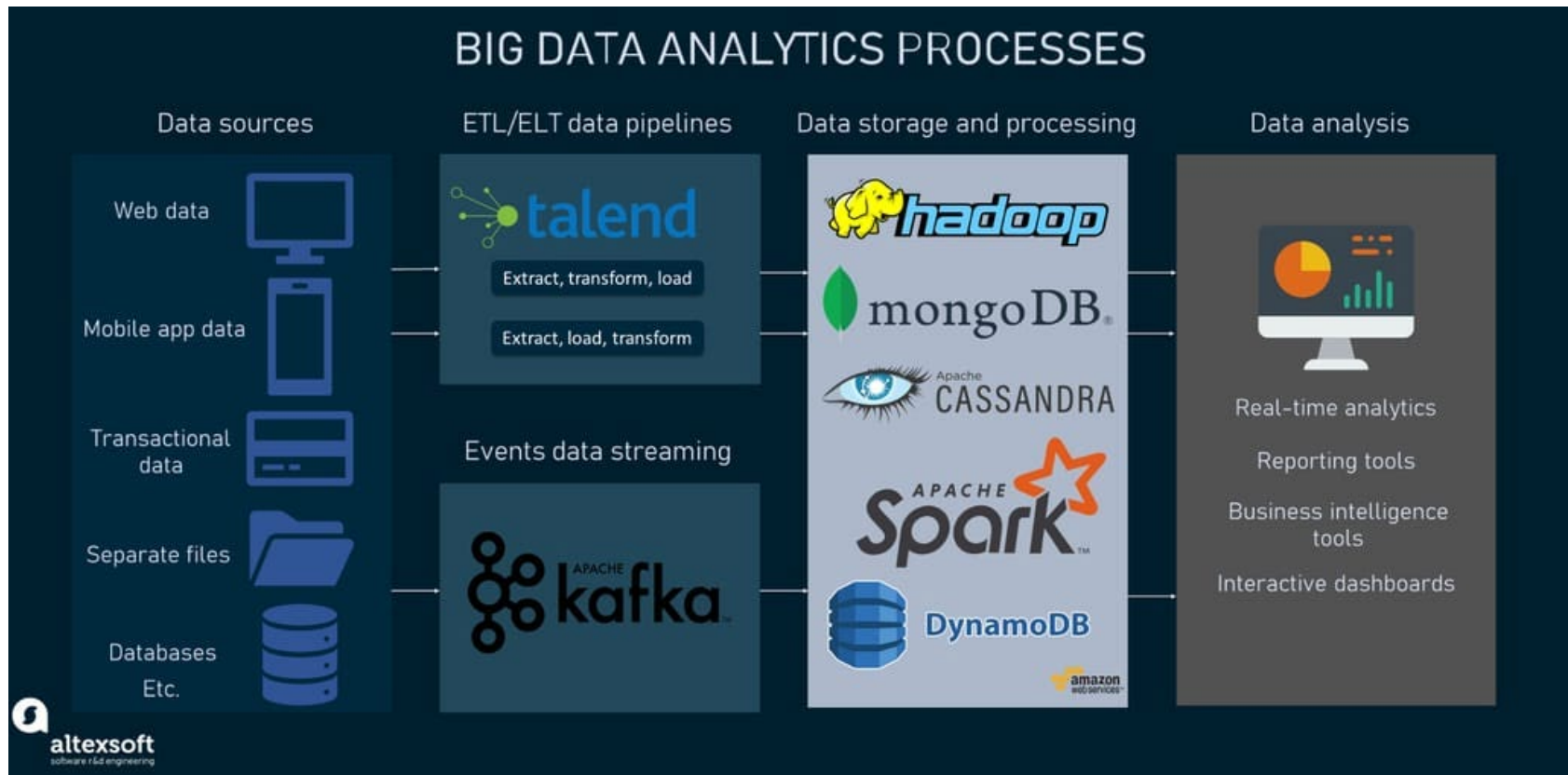
# Hadoop Ecosystem



Src: <https://www.analyticsvidhya.com/blog/2020/10/introduction-hadoop-ecosystem/>

© Walid Hassan, M.B.A, D.Eng.

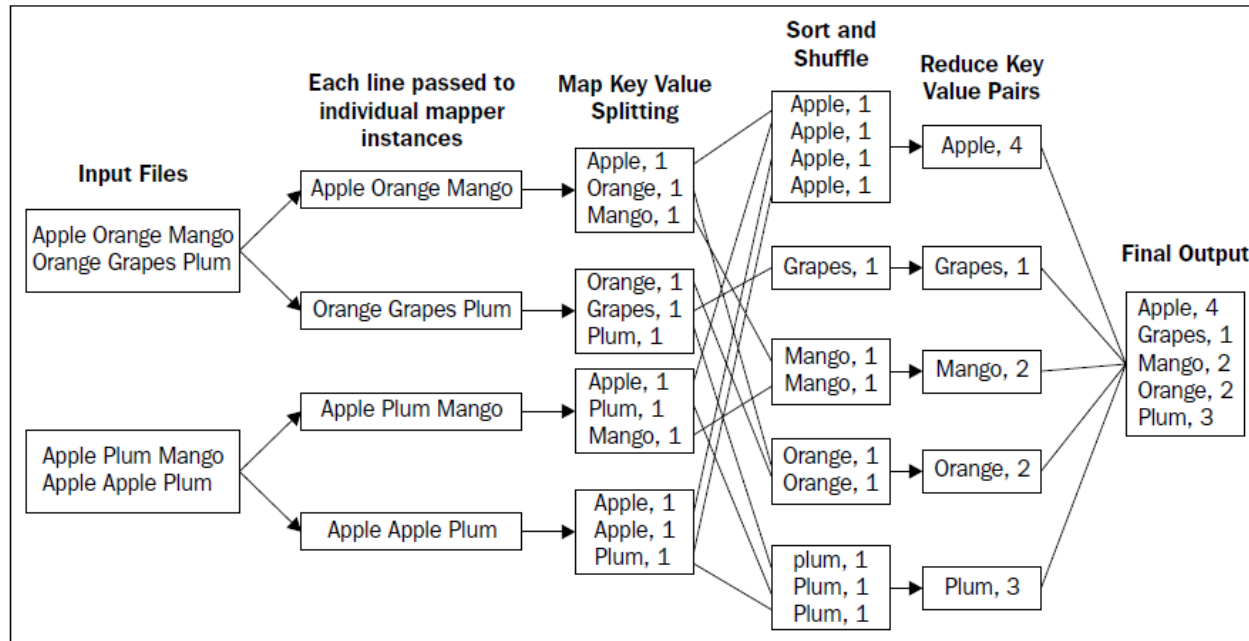
# Big Data Processing



Src: <https://www.altexsoft.com/blog/big-data-analytics-explained/>

© Walid Hassan, M.B.A, D.Eng.

# What is MapReduce in Hadoop?



# What is MapReduce in Hadoop?

1. Each block will be processed. Each line in the block will be sent as an input to the process. This process is called as **Mapper**.
  - Mapper parses the line, gets a word and sets  $\langle \text{word}, 1 \rangle$  for each word.
  - In this example, the output of Mapper for a line Apple Orange Mango will be  $\langle \text{Apple}, 1 \rangle$ ,  $\langle \text{Orange}, 1 \rangle$ , and  $\langle \text{Mango}, 1 \rangle$ . All Mappers will have key as word and value as 1.

# What is MapReduce in Hadoop?

2. The next phase is where the output of Mapper, which has the same key will be consolidated. So key with Apple, Orange, Mango, and others will be consolidated, and values will be appended as a list, in this case <Apple, List<1, 1, 1, 1>>, <Grapes, List<1>>, <Mango, List<1,1>>, and so on.
- The key produced by Mappers will be compared and sorted. This step is called shuffle and sort. The key and list of values will be sent to the next step in the sorted sequence of the key.

# What is MapReduce in Hadoop?

3. The next phase will get  $\langle \text{key}, \text{List}\langle \rangle \rangle$  as input, and will just count the number of 1s in the list and will set the count value as output. This step is called as Reducer, for example, the output for the certain steps are given as follows:

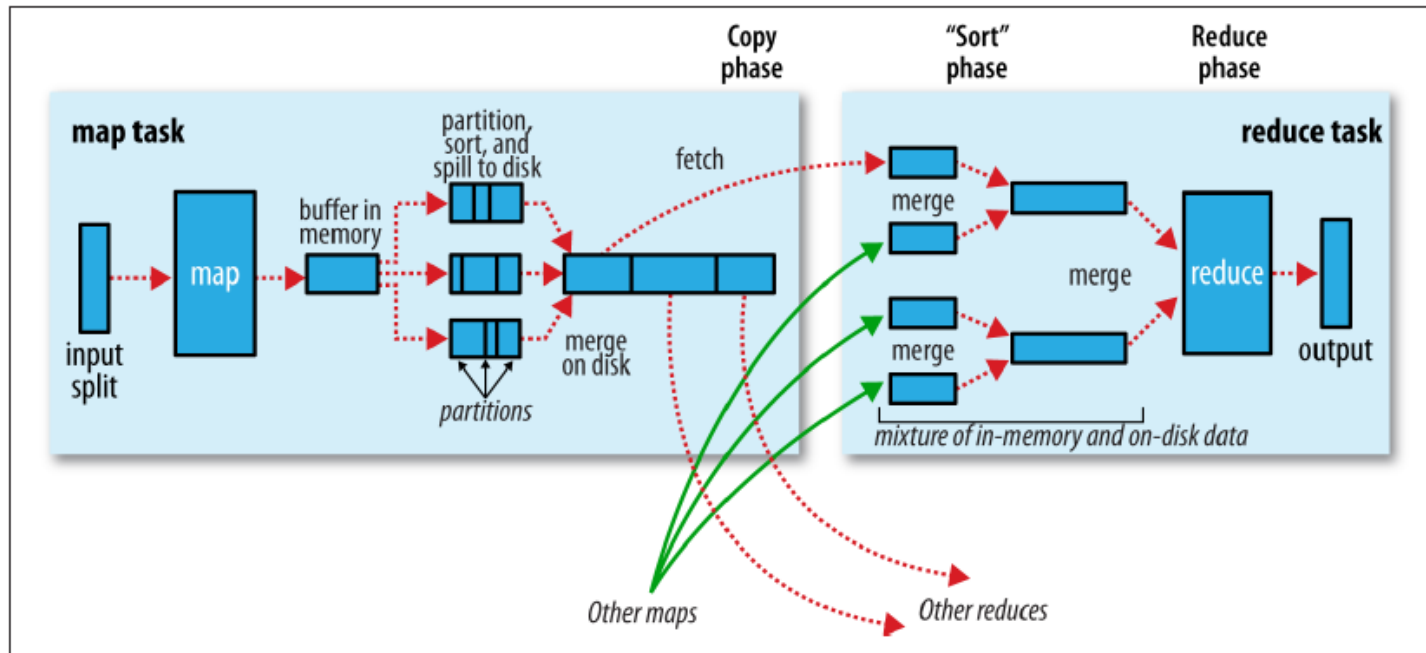
- °°  $\langle \text{Apple}, \text{List}\langle 1, 1, 1, 1 \rangle \rangle$  will be  $\langle \text{Apple}, 4 \rangle$

- °°  $\langle \text{Grapes}, \text{List}\langle 1 \rangle \rangle$  will be  $\langle \text{Grapes}, 1 \rangle$

- °°  $\langle \text{Mango}, \text{List}\langle 1, 1 \rangle \rangle$  will be  $\langle \text{Mango}, 2 \rangle$

4. The Reducer output will be consolidated to a file and will be saved in HDFS as the final output.

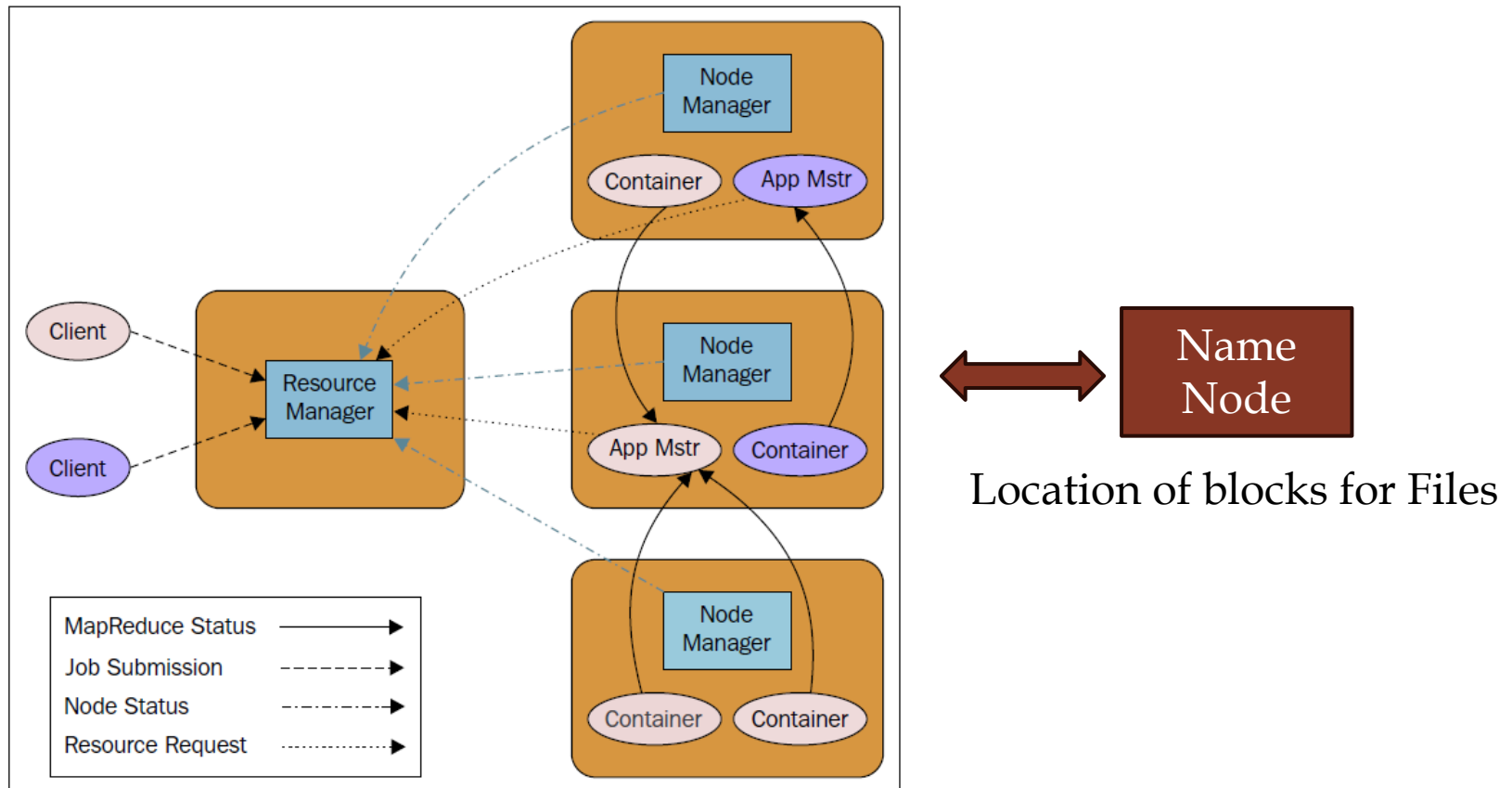
# What is MapReduce in Hadoop?



Src: Hadoop the definitive guide, Tom White, O'Reilly

© Walid Hassan, M.B.A, D.Eng.

# Rack Awareness & Replication

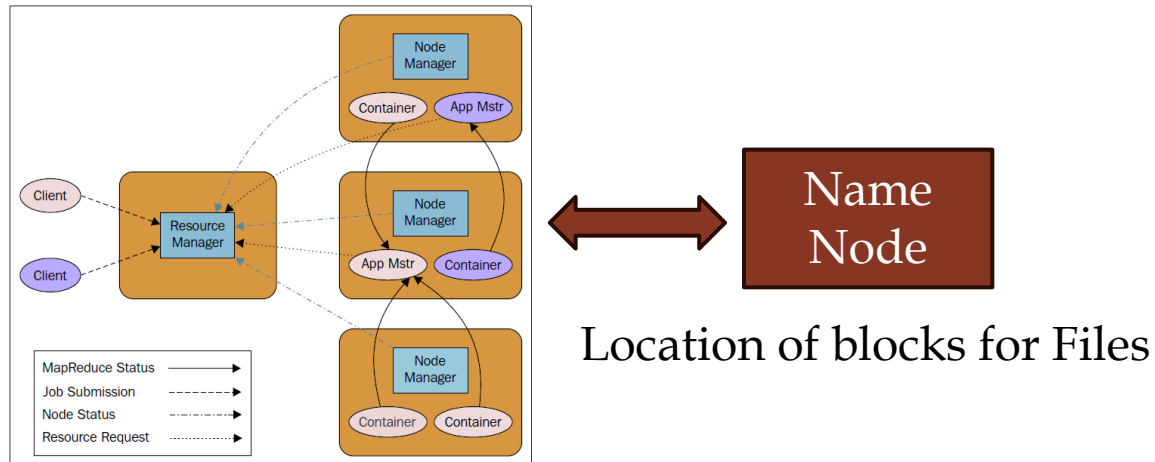


Src:: Hadoop Explained, Aravind Shenoy

© Walid Hassan, M.B.A, D.Eng.



# Rack Awareness & Replication



Application Master (App Master) is responsible for managing the execution of an application and its tasks, including requesting containers from the Resource Manager and coordinating with the Node Managers to launch these containers

# Hadoop File vs. Logical Partitioning....

| Aspect                 | Hadoop File Blocks           | Logical Partitioning                  |
|------------------------|------------------------------|---------------------------------------|
| Purpose                | Storage and fault tolerance. | Optimized data access and processing. |
| Scope                  | Fixed-size chunks of data.   | Data grouped by logical rules.        |
| Storage                | Physical chunks on disk.     | Conceptual and not tied to storage.   |
| Relation to Processing | Transparent to applications. | Directly influences task execution.   |
| Example                | Block size: 128 MB.          | Partitioned by year or region.        |

# What is a file?

- A file is a data entity with specific read, write, and reference characteristics used by software and operating systems. We define a file to have the following characteristics:
  - **Finite length**
    - A file is a finite-length stream of bytes.
  - **Append operations**
    - We can append bytes to the file up to the limits of the host storage system.
  - **Random access**
    - We can read from any location in the file or write updates to any location.

Usually File storage systems organize files into a directory tree.

# File Formats (Parquet)

- it's cross platform
- it's a recognized file format used by many systems
- it stores data in a column layout
- it stores metadata



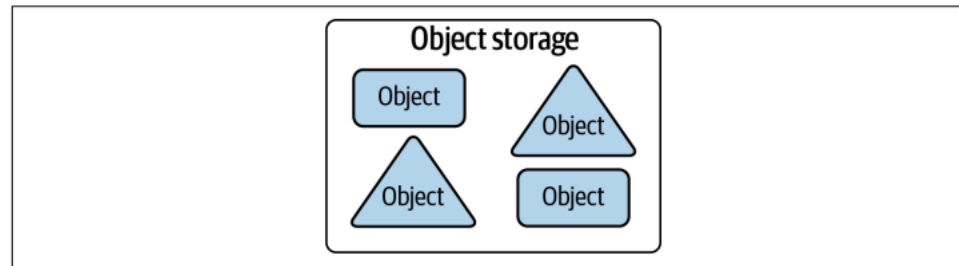
Basis for Delta lake... will be discussed shortly

Src: <https://www.jumpingrivers.com/blog/parquet-file-format-big-data-r/>

© Walid Hassan, M.B.A, D.Eng.

# What is an object?

- The term object storage is somewhat confusing because object has several meanings in computer science. In this context, we're talking about a specialized file-like construct. It could be any type of file – TXT, CSV, JSON, images, videos, or audio.



# Object Storage Characteristics

- One of the most attractive characteristics of cloud object storage is that it is straightforward to manage and use.
- Object storage was arguably one of the first “serverless” services; engineers don’t need to consider the characteristics of underlying server clusters or disks.
- An object store is a key-value store for immutable data objects. We lose much of the writing flexibility we expect with file storage on a local disk in an object store.
- Objects don’t support random writes or append operations; instead, they are written once as a stream of bytes.

# Object Storage

- After this initial write, objects become immutable.
- To change data in an object or append data to it, we must rewrite the full object. Object stores generally support random reads through range requests, but these lookups may perform much worse than random reads from data stored on an SSD.
- Compared to local random access file storage, object stores don't need to support locks or change synchronization, allowing data storage across massive disk clusters.

# Object Storage

- Object stores support extremely performant parallel stream writes and reads across many disks (Transparent).
- In a cloud environment, write / read speed scale based on quotas/available capacity.
- object storage ideal for serving high-volume web traffic or delivering data to highly parallel distributed query engines.
- Typical cloud object stores save data in several availability zones



# Object Storage

- object stores are not good for updates, but this is only partially true. Object stores are an inferior fit for transactional workloads (better use transactional DBs)
- Object stores are now the gold standard of storage for data lakes.

# Amazon S3

- Amazon Simple Storage Service (Amazon S3) is an object storage service that offers industry-leading scalability, data availability, security, and performance.
- Customers of all sizes and industries can use Amazon S3 to store and protect any amount of data for a range of use cases, such as data lakes, websites, mobile applications, backup and restore, archive, enterprise applications, IoT devices, and big data analytics.

# Amazon S3



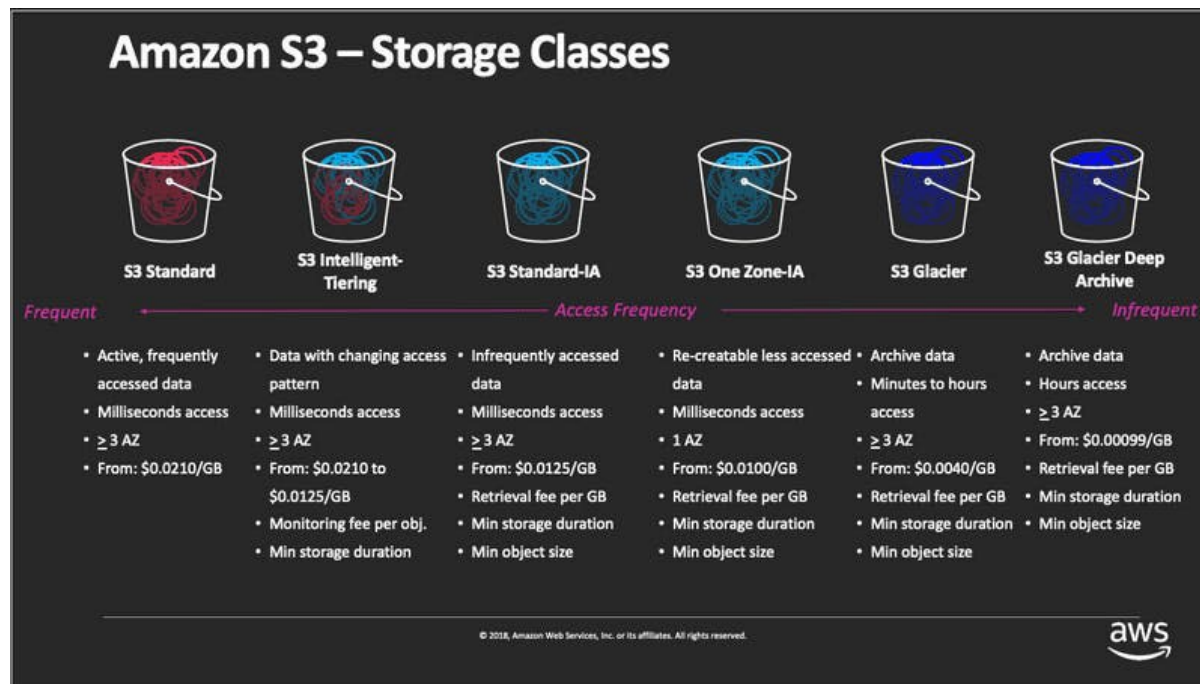
Src: amazon

© Walid Hassan, M.B.A, D.Eng.

# Amazon S3

## Storage classes:

Amazon S3 offers a range of storage classes designed for different use cases. For example, you can store mission-critical production data in S3 Standard or S3 Express One Zone for frequent access.



Src: amazon; <https://blog.jineshkumar.com/all-about-aws-s3-storage>

© Walid Hassan, M.B.A, D.Eng.

# Amazon S3

| AWS S3 STORAGE CLASSES        |               |                                                           |                                                           |                    |  |
|-------------------------------|---------------|-----------------------------------------------------------|-----------------------------------------------------------|--------------------|-------------------------------------------------------------------------------------|
| Features                      | S3-Standard   | S3-Standard IA                                            | S3 One Zone IA                                            | S3 - RRS           | Glacier                                                                             |
| Durability                    | 99.999999999% | 99.999999999%                                             | 99.999999999%<br>(Single AZ)                              | 99.99%             | 99.999999999%                                                                       |
| Availability                  | 99.99%        | 99.9%                                                     | 99.5%                                                     | 99.99%             | NA                                                                                  |
| Availability Zones            | >=3           | >=3                                                       | 1                                                         | 1                  | >=3                                                                                 |
| Minimum Storage Duration      | Unlimited     | 30 Days                                                   | 30 Days                                                   | Unlimited          | 90 Days                                                                             |
| Access Speed (Latency)        | Miliseconds   | Miliseconds                                               | Miliseconds                                               | Miliseconds        | 1 min to 12 hrs                                                                     |
| Data Retrieval Fee            | Nil           | Per GB Data Retrived                                      | Per GB Data Retrived                                      | Nil                | Per GB Data Retrived                                                                |
| Minimum Billable Object Size  | NA            | 128 KB<br>(the smaller objects will be charged for 128KB) | 128 KB<br>(the smaller objects will be charged for 128KB) | NA                 | NA                                                                                  |
| Type of Storage               | Object        | Object                                                    | Object                                                    | Object             | Object                                                                              |
| Lifecycle Management Policies | Available     | Available                                                 | Available                                                 | Available          | Available                                                                           |
| SSL Support                   | Yes           | Yes                                                       | Yes                                                       | Yes                | Yes                                                                                 |
| Amazon S3 SLA Backing         | 99.9%         | 99%                                                       | 99%                                                       | Data Not Available | NA                                                                                  |

Src: amazon; <https://www.whizlabs.com/blog/aws-s3/>

© Walid Hassan, M.B.A, D.Eng.

# Amazon S3 – Storage Management

- **S3 Lifecycle** – Configure a lifecycle configuration to manage your objects and store them cost effectively throughout their lifecycle. You can transition objects to other S3 storage classes or expire objects that reach the end of their lifetimes.
- **S3 Object Lock** – Prevent Amazon S3 objects from being deleted or overwritten for a fixed amount of time or indefinitely (regulatory/extra protection).
- **S3 Replication** – Replicate objects and their respective metadata and object tags to one or more destination buckets in the same or different AWS Regions for reduced latency, compliance, security, and other use cases.

# Object Storage vs. File Storage

- In traditional block storage systems, data is divided into blocks (e.g., 512 KB or 4 KB in size), with the disk controller managing the addresses and locations of each block.
- S3, as an object storage system, abstracts away blocks and sectors. It does not treat data as a series of fixed-size blocks, but rather as entire objects that can vary in size, up to a maximum of 5 TB for a single object.

Src: amazon

© Walid Hassan, M.B.A, D.Eng.



# How S3 Works

- Amazon S3 is an object storage service that stores data as objects within buckets.

An object is a file and any metadata that describes the file.

- A bucket is a container for objects. You can store any number of objects in a bucket and can have up to 100 buckets in your account.
- To store your data in Amazon S3, you first create a bucket and specify a bucket name and AWS Region. Then, you upload your data to that bucket as objects in Amazon S3. Each object has a key(or key name), which is the unique identifier for the object within the bucket.



# How S3 Works

- S3 provides features that you can configure to support your specific use case. For example, you can use S3 Versioning to keep multiple versions of an object in the same bucket, which allows you to restore objects that are accidentally deleted or overwritten.
- Buckets and the objects in them are private and can be accessed only if you explicitly grant access permissions. You can use bucket policies, AWS Identity and Access Management (IAM) policies, access control lists (ACLs), and S3 Access Points to manage access.

# Objects

- Objects consist of object data and metadata. The metadata is a set of name-value pairs that describe the object. These pairs include some default metadata, such as the date last modified, and standard HTTP metadata, such as Content-Type
- An object is uniquely identified within a bucket by a key (name) and a version ID (if S3 Versioning is enabled on the bucket).

# Keys

- An object key (or key name) is the unique identifier for an object within a bucket.

Every object in a bucket has exactly one key. The combination of a bucket, object key, and optionally, version ID (if S3 Versioning is enabled for the bucket) uniquely identify each object. So you can think of Amazon S3 as a basic data map between "bucket + key + version" and the object itself.

# Regions

- You can choose the geographical AWS Region where Amazon S3 stores the buckets that you create.
- You might choose a Region to optimize latency, minimize costs, or address regulatory requirements.
- Objects stored in an AWS Region never leave the Region unless you explicitly transfer or replicate them to another Region

# Amazon Data Consistency Model

- Amazon S3 provides strong read-after-write consistency for PUT and DELETE requests of objects in your Amazon S3 bucket in all AWS Regions.
- This behavior applies to both writes to new objects as well as PUT requests that overwrite existing objects and DELETE requests. In addition, read operations on Amazon S3 Select, Amazon S3 access controls lists (ACLs), Amazon S3 Object Tags, and object metadata (for example, the HEAD object) are strongly consistent.
- Updates to a single key are atomic.
- Amazon S3 achieves high availability by replicating data across multiple servers within AWS data centers

Src: amazon

© Walid Hassan, M.B.A, D.Eng.

# Strong Consistency

- **Assurance of Correctness:** Strong consistency is essential in applications where it's critical that every read operation returns the most recent write. This is crucial for scenarios where correctness is non-negotiable, like financial transactions, inventory management, or critical system logs.
- **Data Integrity Across Nodes:** In systems like Amazon S3, strong consistency ensures that data reads immediately reflect any successful write or update, regardless of which node is serving the read request.
- **Drawbacks:** Strong consistency often requires **synchronization across nodes** or **quorum-based reads and writes**.

Src: amazon

© Walid Hassan, M.B.A, D.Eng.

# Eventual Consistency

- **High Availability and Low Latency:** Eventual consistency provides high availability and lower latency by allowing nodes to respond to read and write requests independently without waiting for synchronization. This is particularly advantageous in systems where scalability and performance are prioritized.
- **Trade-Off with Stale Reads:** there's a possibility of reading outdated data immediately after a write, as updates propagate asynchronously. However, it's guaranteed that given enough time without further updates, all nodes will converge to the latest value.
- **Use Cases:** Eventual consistency is suitable for applications where absolute correctness is not required immediately, such as social media feeds, caching layers, and some content delivery networks (CDNs)



Hands On.....

Amazon S3



# S3 Demo

[Amazon S3](#) > [Buckets](#) > [whawsai](#) > Edit Object Ownership

## Edit Object Ownership [Info](#)

### Object Ownership

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

☒ **ACLs disabled (recommended)**

All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ **ACLs enabled**

Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

Object Ownership

Bucket owner enforced

Cancel

Save changes

Src: amazon

© Walid Hassan, M.B.A, D.Eng.

# S3 Demo

This XML file does not appear to have any style information associated with it. The document tree is shown below.

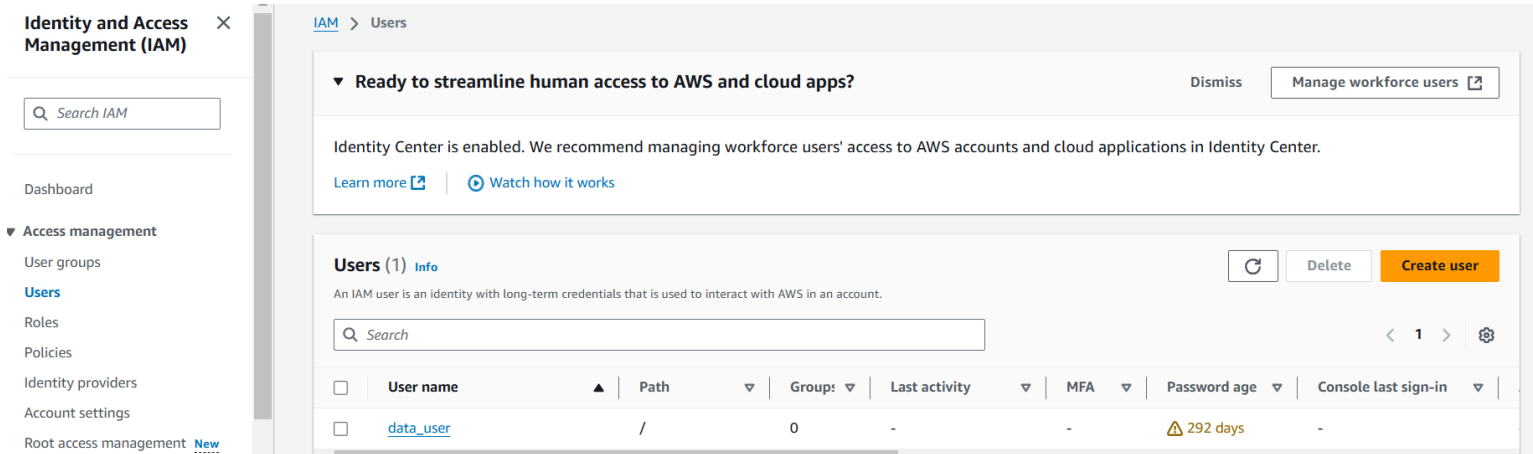
```
▼ <Error>  
  <Code>AccessDenied</Code>  
  <Message>Access Denied</Message>  
  <RequestId>ATYMRKJ80HQNKHC4</RequestId>  
  <HostId>KIS2aWxuSVjSodbpXmnPTHFdNMWj8biI+FwsRXX94/TYK5km8XyOECvLRHwuqy7cJ5XJZ4eu6aL++POXZpHjp9F/e1MUAJOeawLizA1F1L4=</HostId>  
</Error>
```

Accessing the objects is not enabled by default... you need to work with access control lists to enable that.

Src: amazon

© Walid Hassan, M.B.A, D.Eng.

# S3 Demo



To access your file thru URI, you need a user/access key...

Src: amazon

© Walid Hassan, M.B.A, D.Eng.

# S3 Demo

[IAM](#) > [Users](#) > data\_user

data\_user [Info](#)

Delete

## Summary

ARN

 arn:aws:iam::351187099241:user/data\_user

Console access

 Enabled without MFA

Access key 1

[Create access key](#)

Created

January 26, 2024, 16:15 (UTC-06:00)

Last console sign-in

 Never. Password enabled 292 days ago

[Permissions](#)

[Groups](#)

[Tags](#)

[Security credentials](#)

[Last Accessed](#)

## Permissions policies (2)

Permissions are defined by policies attached to the user directly or through groups.



Remove

Add permissions ▼

To access your file thru URI, you need a user/access key...

Src: amazon

© Walid Hassan, M.B.A, D.Eng.

# S3 Demo

## Use case

☐ **Command Line Interface (CLI)**

You plan to use this access key to enable the AWS CLI to access your AWS account.

☐ **Local code**

You plan to use this access key to enable application code in a local development environment to access your AWS account.

☐ **Application running on an AWS compute service**

You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.

☐ **Third-party service**

You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.

☐ **Application running outside AWS**

You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.

☐ **Other**

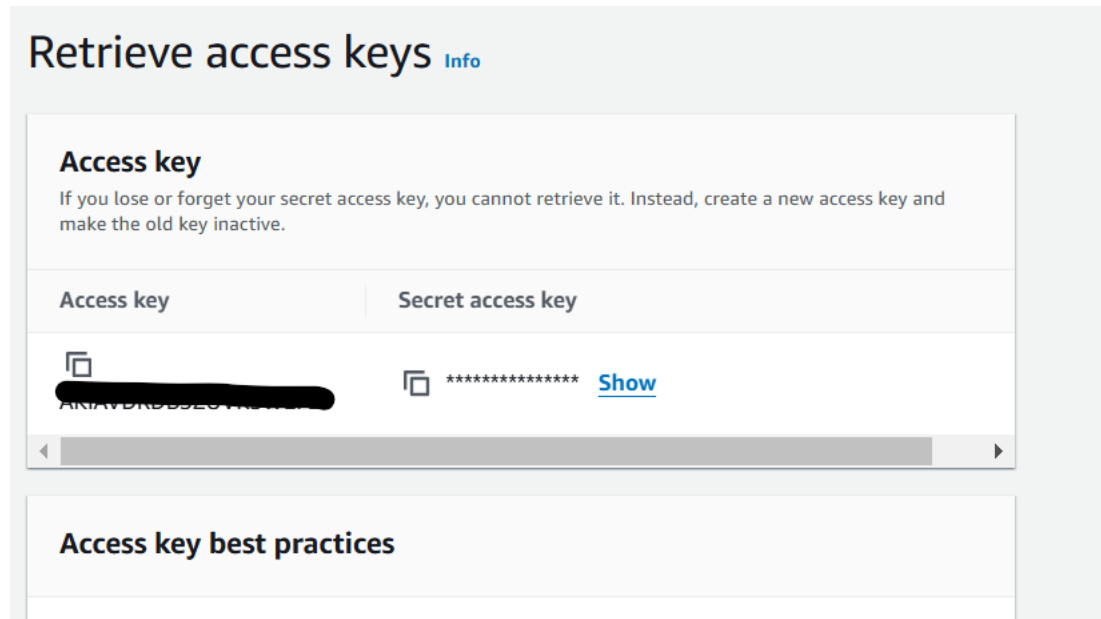
Your use case is not listed here.

To access your file thru URI, you need a user/access key...

Src: amazon

© Walid Hassan, M.B.A, D.Eng.

# S3 Demo

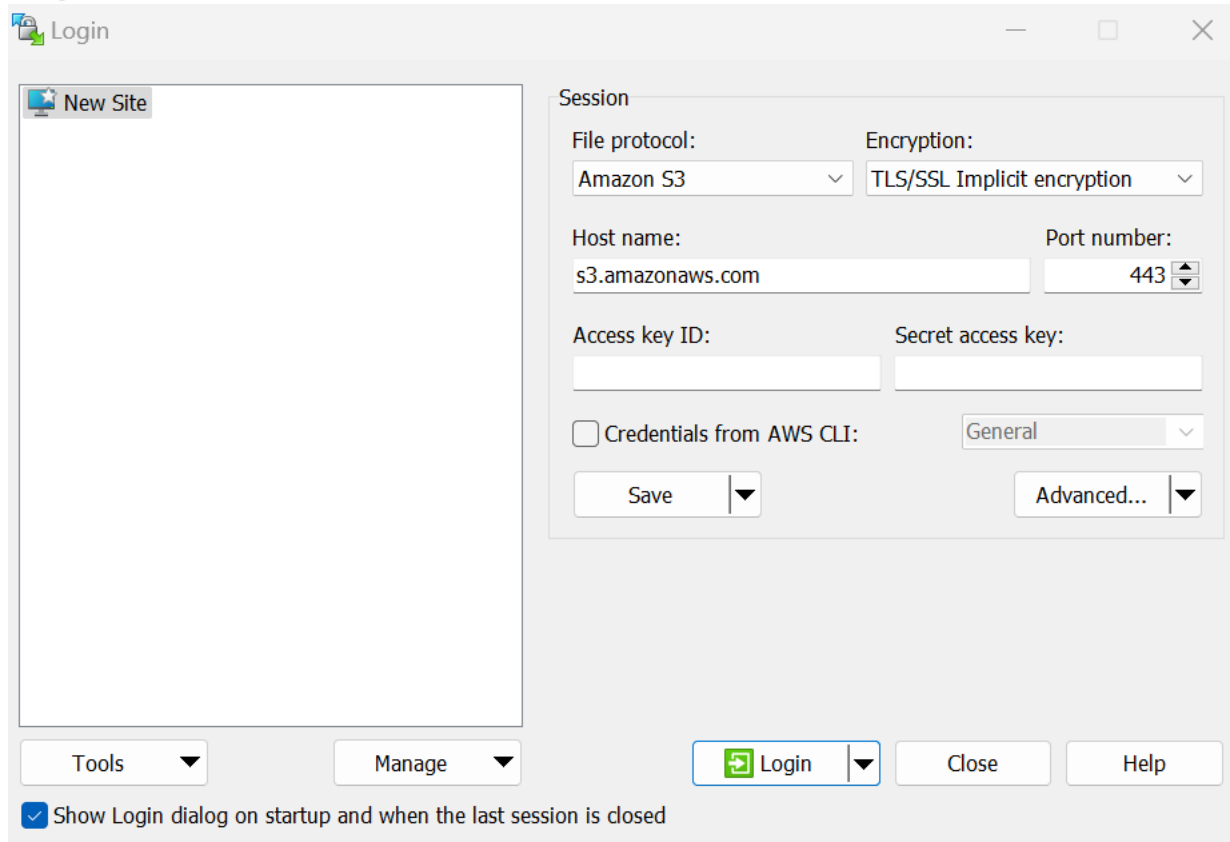


To access your file thru URI, you need a user/access key...

Src: amazon

© Walid Hassan, M.B.A, D.Eng.

# S3 Demo



The screenshot shows a 'Login' dialog box with a 'New Site' tab. The 'Session' section contains the following fields:

- File protocol: Amazon S3 (dropdown)
- Encryption: TLS/SSL Implicit encryption (dropdown)
- Host name: s3.amazonaws.com (text field)
- Port number: 443 (spin box)
- Access key ID: (empty text field)
- Secret access key: (empty text field)
- ☐ Credentials from AWS CLI: (checkbox)
- General (dropdown menu)
- Save (button)
- Advanced... (button)

At the bottom, there are buttons for Tools, Manage, Login (with a green arrow icon), Close, and Help. A checkbox labeled 'Show Login dialog on startup and when the last session is closed' is checked.

To access your file thru URI, you need a user/access key...

Src: amazon

© Walid Hassan, M.B.A, D.Eng.

# S3 Demo in Databricks

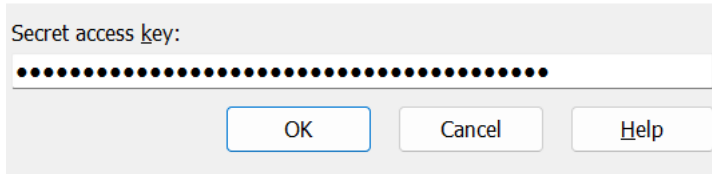
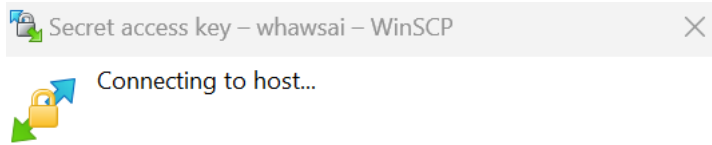
To access your file thru URI, you need a user/access key...

Src: amazon

© Walid Hassan, M.B.A, D.Eng.



# S3 Demo



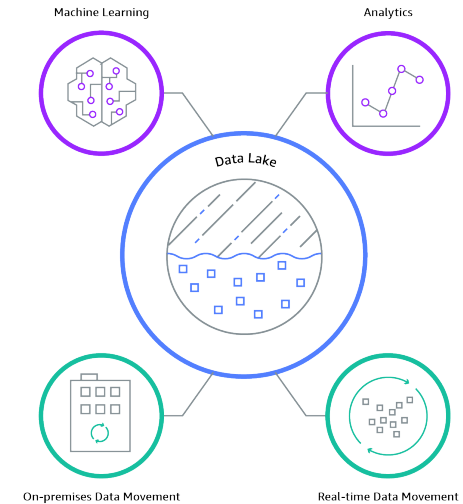
To access your file thru URI, you need a user/access key...

Src: amazon

© Walid Hassan, M.B.A, D.Eng.

# What is a Data Lake?

- A data lake is a repository that enables you to store all the structured and unstructured data you collected at any scale.
- You can store your data as-is with data lakes without having first to structure the data.
- Data lakes allow you to run different types of analytics that can help you make better decisions.



Src: <https://cmcglobal.com.vn/digital-transformation/data-analytics-ai/what-are-data-lakes/>

© Walid Hassan, M.B.A, D.Eng.

# What is a Data Lake?

**Metadata** -- Responsible for maintaining data schema. Can be leveraged to increase computational efficiency and enhance useability.



**Compute** -- Responsible for performing crunching of data. Compute engines can either come coupled with storage like in a warehouse, or focus solely on compute.



**Format** -- The shape or format data takes in the storage layer. Format influences available operations and compute engine compatibility



**Storage** -- Where data physically resides. Requirements include durability, scalability, and interoperability with other tools, engines, and libraries.



Src: <https://cmcglobal.com.vn/digital-transformation/data-analytics-ai/what-are-data-lakes/>

© Walid Hassan, M.B.A, D.Eng.

# Characteristics of a Data Lake

- **Raw Data Storage:** Data lakes store raw data in its native format, whether it's JSON files, CSVs, images, videos, or logs. This allows flexibility for future use cases that may not yet be defined.
- **Scalable and Cost-Effective:** They are built to handle vast amounts of data and are typically implemented on scalable and cost-efficient storage systems like Amazon S3, Azure Data Lake, or Hadoop-based solutions.
- **Schema-on-Read:** Unlike data warehouses, which enforce a schema upfront (schema-on-write), data lakes allow users to define the schema when the data is read, making it more flexible for exploratory analysis.

# Data Lake Challenges

- Data lakes revolutionized data analysis.
- Challenges:
  - Multiple copies of the same data often enter a data lake in different locations (big problem without data governance).
  - Increased costs of redundant storage and because multiple copies introduced the potential for inconsistencies → a considerable amount of research was needed to determine the lineage of each copy.

Src: Databricks Lakehouse Platform Cookbook, Dr. Alan L. Dennis

© Walid Hassan, M.B.A, D.Eng.

# Other Data Lake Challenges

- With data lakes, by definition, we load data into the lake and then clean and refine it as the data is needed to address business problems or questions.
- This iterative refinement approach introduces the potential of using data that has not been cleansed while believing it has been.
- Zones (or layers of refinement) were introduced to address this issue, they often relied on the data's location in the lake as it was the only indicator of the quality of the data.
- To address these challenges, many organizations utilize data catalogs to record the quality of data sets.

Src: Databricks Lakehouse Platform Cookbook, Dr. Alan L. Dennis

© Walid Hassan, M.B.A, D.Eng.

# Other Data Lake Challenges

- Data is often stored in files that do not natively support schemas. While this flexibility is useful, the data format can differ when dealing with each dataset.
- To use the data, one must first determine the dataset's location in a complex directory structure and then determine the file format in which the data is stored.
- Next, the user must determine the schema of the data. This operation can be difficult with formats that support complex data storage, such as JSON.

Src: Databricks Lakehouse Platform Cookbook, Dr. Alan L. Dennis

© Walid Hassan, M.B.A, D.Eng.

# What is a Data Warehouse

- A data warehouse is a central repository of information that can be analyzed to make more informed decisions.
- Data flows into a data warehouse from transactional systems, relational databases, and other sources, typically on a regular cadence.
- Business analysts, data engineers, data scientists, and decision makers access the data through business intelligence (BI) tools, SQL clients, and other analytics application.

Src: <https://aws.amazon.com/what-is/data-warehouse/>

© Walid Hassan, M.B.A, D.Eng.



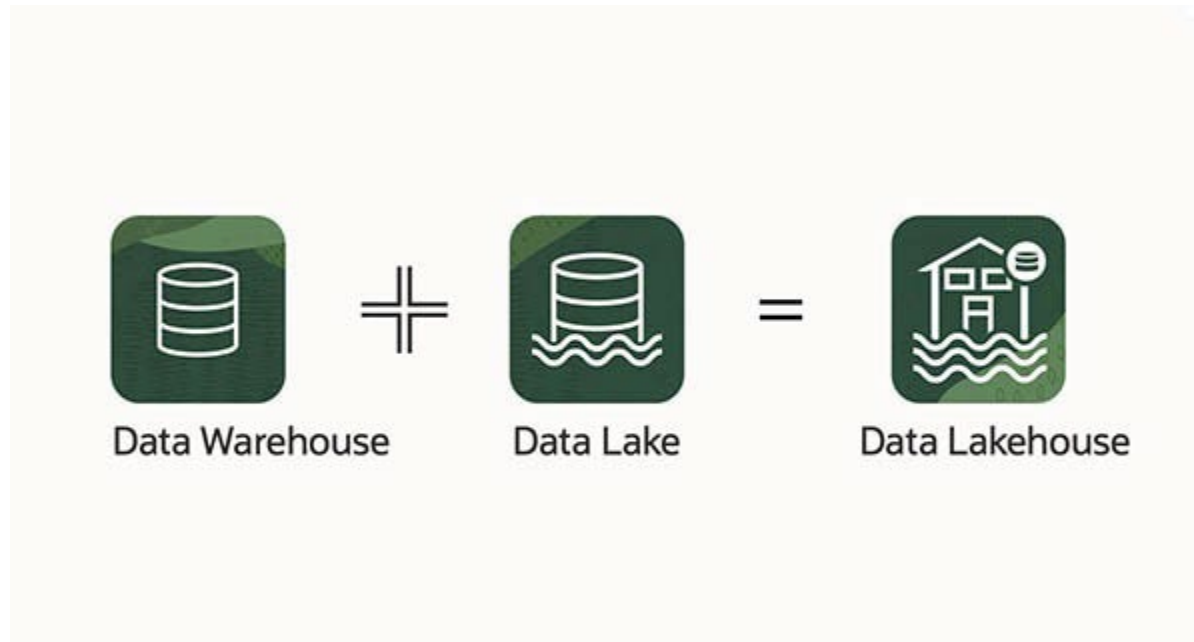
# DW Challenges

- When data is in a data warehouse, it has been through extensive processing before its being made available.
- Data warehouses are highly governed and, in turn, highly rigid.
- This characteristic is designed to ensure that the information within the data warehouse is correct and can be used to make business decisions → changes to data warehouses are difficult and highly governed.

Src: Databricks Lakehouse Platform Cookbook, Dr. Alan L. Dennis

© Walid Hassan, M.B.A, D.Eng.

# Data Lakehouse



Best of Both Worlds....

Src: <https://www.oracle.com/big-data/what-is-data-lakehouse/>

© Walid Hassan, M.B.A, D.Eng.

# Data Lakehouse

“ A data lakehouse can be defined as a modern data platform built from a combination of a data lake and a data warehouse. More specifically, a data lakehouse takes the flexible storage of unstructured data from a data lake and the management features and tools from data warehouses, then strategically implements them together as a larger system. This integration of two unique tools brings the best of both worlds to users.”

Src: <https://www.oracle.com/big-data/what-is-data-lakehouse/>

© Walid Hassan, M.B.A, D.Eng.

# Data Lakehouse

- A data lakehouse addresses these challenges by layering schemas and constraints on top of a data lake.
- Data are organized into zones with unambiguous names. When a data item is part of a zone, such as Silver, that designation has a clear and consistent implication.
- Lakehouse combines structured, semi-structured, and unstructured data. This single location for data enables users with different goals to use the same data store.

Src: Databricks Lakehouse Platform Cookbook, Dr. Alan L. Dennis

© Walid Hassan, M.B.A, D.Eng.

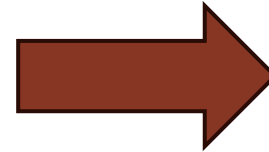
# Data Lakehouse Improvements

- Have a single processing path.
- Rather than separate flows of data into different paths based on their refresh rate, data is processed in a single flow (Ideally, data is processed incrementally).
- If the data must be accessed quickly, incremental processing may occur continuously, leveraging the micro-batch element of spark structured streaming.

# Delta Lake

As lakehouse takes the best of two worlds:

- From data lakes: Scalable, cost-efficient storage of raw data and support for diverse data types.
- From data warehouses: Reliability, ACID transactions, and performance for analytics and structured queries



Databricks Offering

Src: Databricks Lakehouse Platform Cookbook, Dr. Alan L. Dennis

© Walid Hassan, M.B.A, D.Eng.

# Delta Lake

Delta Lake is a relatively new addition to the data field, and Databricks created Delta in 2019. Rather than keeping it proprietary, Databricks open sourced the project, enabling an open and collaborative approach



Src: Databricks Lakehouse Platform Cookbook, Dr. Alan L. Dennis

© Walid Hassan, M.B.A, D.Eng.

Delta Lake is Databricks' technology that implements the lakehouse paradigm by adding features like transactionality and schema enforcement to data lakes, making them suitable for both raw data storage and analytics workloads. Here's how it fits the lakehouse vision:

- **ACID Transactions:** Delta Lake brings full ACID compliance to data lakes, ensuring consistency and reliability in data operations.
- **Unified Architecture:** It enables data lakes to handle both raw, unstructured data (like a data lake) and structured, queryable data (like a data warehouse).
- **Time Travel:** Built-in support for versioning allows users to "time travel" through historical data states, making debugging and auditing easier.

Src: Databricks Lakehouse Platform Cookbook, Dr. Alan L. Dennis



# What is ACID in DB systems



ACID is an acronym that describes the key properties of reliable database transactions.

## A: Atomicity

- Ensures that a transaction is "all or nothing."
- If any part of the transaction fails, the entire transaction is rolled back, leaving the database in its original state.

## C: Consistency

- Guarantees that a transaction will bring the database from one valid state to another.
- It ensures that all rules, constraints, and integrity checks (like foreign key constraints) are maintained.

Src: Databricks Lakehouse Platform Cookbook, Dr. Alan L. Dennis

© Walid Hassan, M.B.A, D.Eng.

# What is ACID in DB systems



## I: Isolation

- Ensures that transactions are executed independently of one another.
- Concurrent transactions do not interfere, preserving the integrity of data (e.g., through locking or multiversion concurrency control).

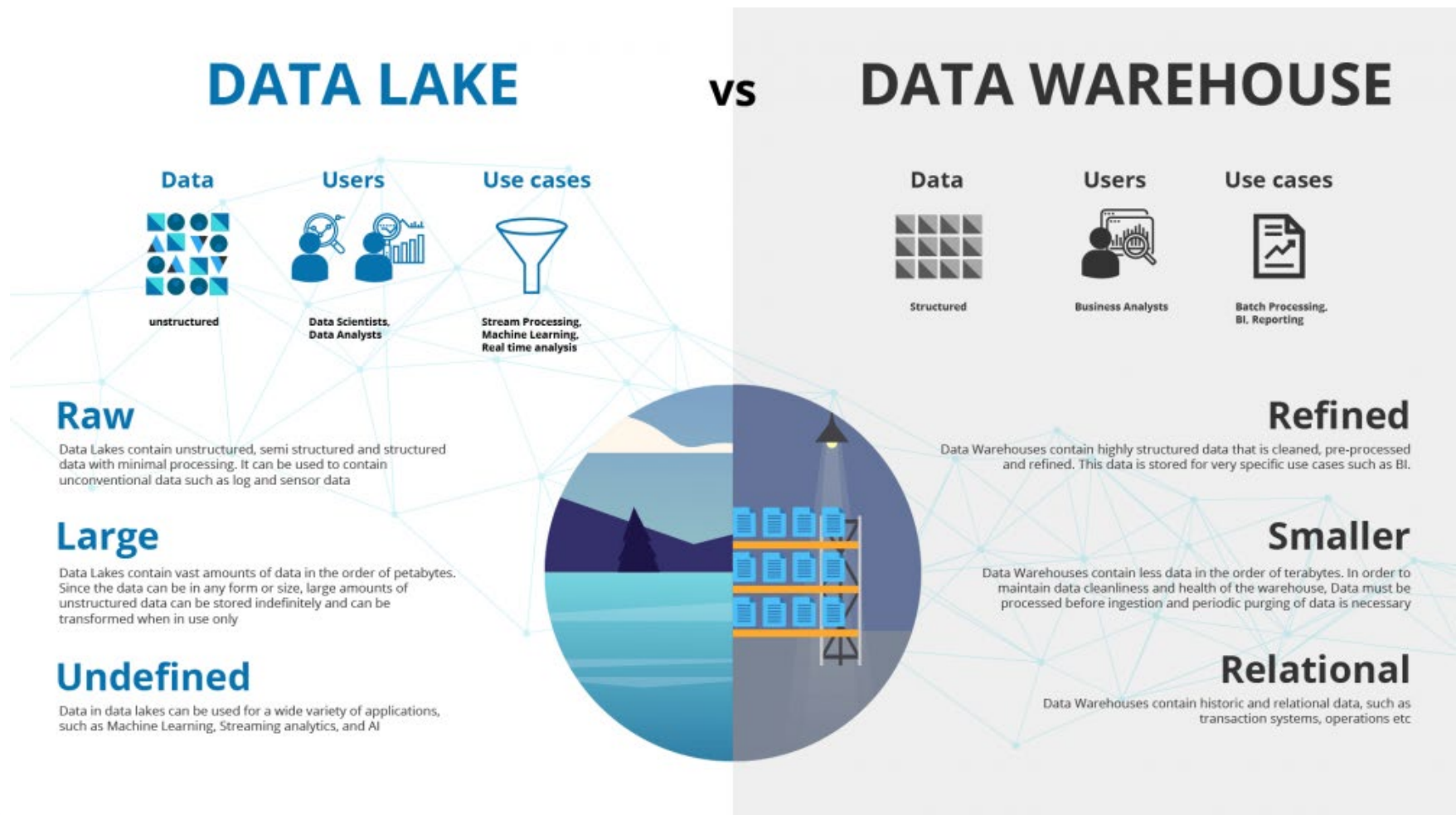
## D: Durability

- Ensures that once a transaction is committed, its changes are permanent, even in the case of a system failure (e.g., power outage or crash).
- This is typically achieved by writing changes to persistent storage.

Src: Databricks Lakehouse Platform Cookbook, Dr. Alan L. Dennis

© Walid Hassan, M.B.A, D.Eng.

# Data Lake vs. Warehouse



Src: <https://medium.com/p/2faaf10db585>

© Walid Hassan, M.B.A, D.Eng.

# References

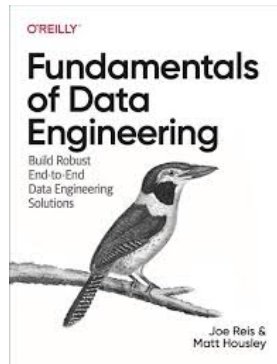
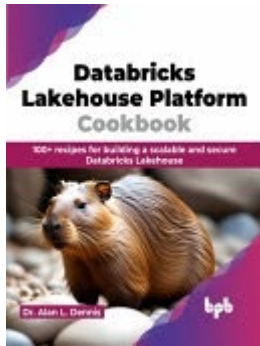
In addition to the references in each slide the below are leveraged throughout this course.

Gareth James, D. W. (2023). An Introduction to Statistical Learning: with Applications in Python. Springer.

(Jules S. Damji, July 2020), Learning Spark Lightning Fast Data Analytics , 2020 (2<sup>nd</sup> Edition).

Databricks Lakehouse Platform Cookbook, Dennis, Dr. Alan L.

Fundamentals of Data Engineering, O'Reilly (J. Reis, M. Housley)



O'REILLY®

# Learning Spark

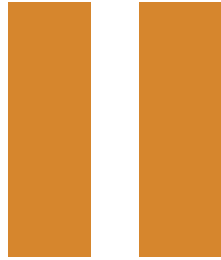
Lightning-Fast Data Analytics

2nd Edition  
Covers  
Apache Spark 3.0



© Walid Hassan, M.B.A, D.Eng.

THE GEORGE  
WASHINGTON  
UNIVERSITY  
WASHINGTON, DC



# BREAK



Please come back @  
2:31 PM EST  
1:31 PM CST

# Vector Databases...

# Word2Vec Embedding...

Bank Embedding is an array of 300 elements in length and floating point

- `<class 'numpy.ndarray'> 300`
- Float32
- Let us take a look...



Word2VecBankEmbedding.txt

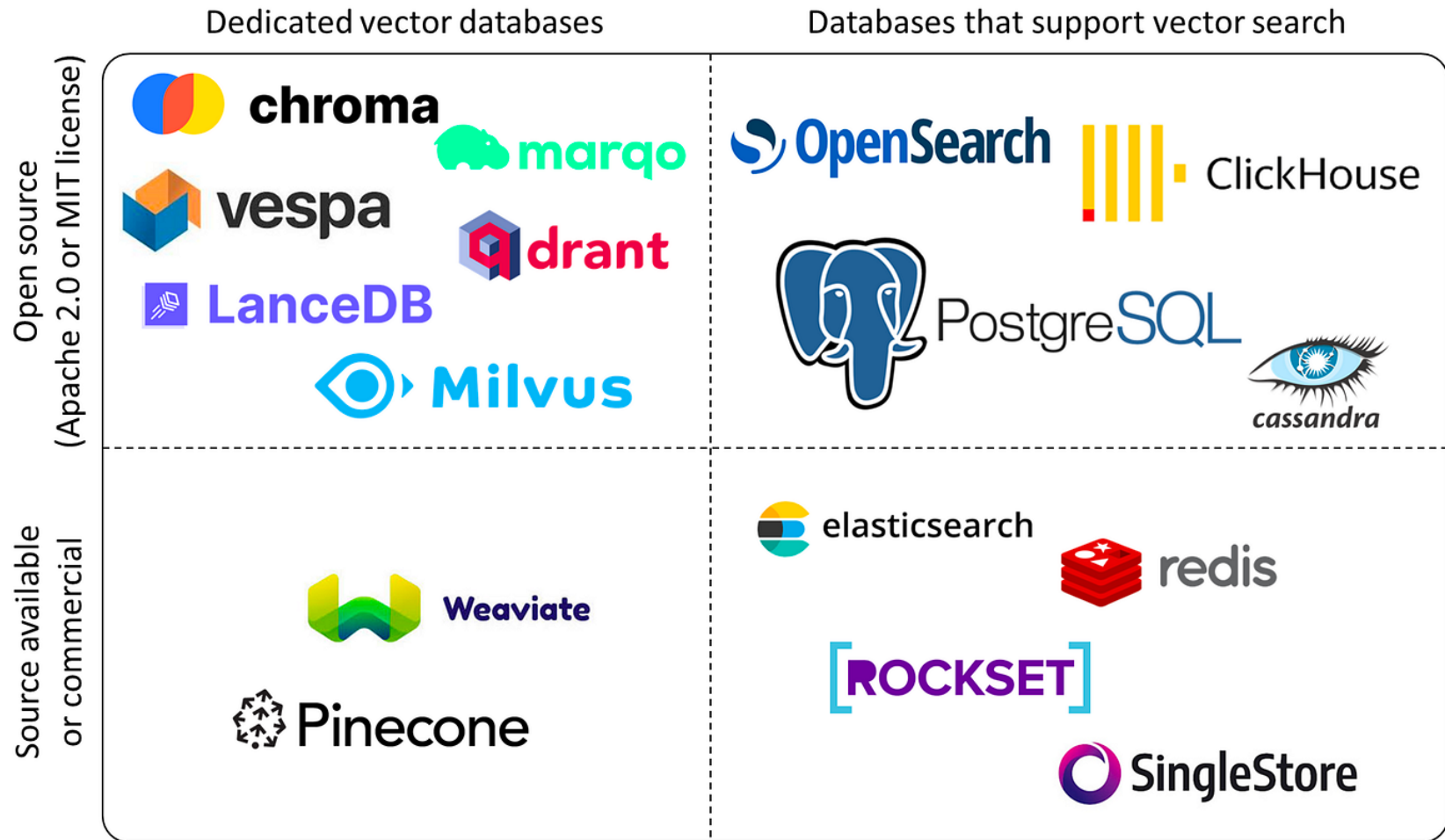
Src: Gareth James, D. W. (2023). An Introduction to Statistical Learning: with Applications in Python. Springer.

Src: Josh Strommer, Statquest: Word Embedding and Word2Vec clearly explained

© Walid Hassan, M.B.A, D.Eng.



# Vector DBs



Src: <https://www.datacamp.com/blog/the-top-5-vector-databases>

© Walid Hassan, M.B.A, D.Eng.



# Vector DBs

**Searching:** One of the primary functions of a vector database is to enable similarity or nearest-neighbor search among embeddings. Given a query vector, the database can quickly find the most similar vectors in its collection. This operation is fundamental in applications like recommendation systems, image recognition, and semantic search.

**Integration with Machine Learning:** Many vector databases integrate with popular machine learning frameworks and libraries. They often support both training-time and inference-time operations, allowing seamless transitions from model development to deployment

Src: Gareth James, D. W. (2023). An Introduction to Statistical Learning: with Applications in Python. Springer.  
Src: <https://airbyte.com/data-engineering-resources/vector-databases>

© Walid Hassan, M.B.A, D.Eng.